Safety-Aware Vehicle Routing for Quick-Commerce Delivery in Indian Cities

An ε-Constraint Formulation with Soft Time Windows

Pranav Bhadane
Roll No. 21IM30017

Department of Industrial and Systems Engineering
Indian Institute of Technology Kharagpur

Supervisor: Prof. Subhajit Sidhanta

M.Tech. Thesis Presentation, 2026

Introduction: The Q-Commerce Routing Problem

Setting

- Platforms like **Blinkit**, Zepto, Instamart promise **10-minute delivery** from neighbourhood dark stores
- Fleet of **two-wheeler riders** in dense Indian cities
- High dispatch frequency, narrow residential streets

The Safety Problem

- India: **4,61,312 road accidents** and **1,68,491 fatalities** in 2022 (MoRTH)
- Fastest route is not the safest route
- Conventional VRP treats safety as an afterthought

Why Existing Models Fail

- Minimise only travel time or distance
- Crash risk as a weighted term $\Rightarrow$ **hidden trade-offs**
- No explicit residential-street protection
- No calibration story for risk estimates

Literature Review

VRPTW with Soft Time Windows

- Solomon (1987): benchmark VRPTW instances
- Koskosidis et al. (1992): soft TW with linear penalty
- Taillard et al. (1997): convex penalties rarely linearised
- Desrochers et al. (1992): exact column generation

Safety-Aware Routing

- Hazmat origins: Zografos & Erkut
- Hoseinzadeh et al. (2020): cumulative risk, *weighted-sum* objective
- Salmon et al. (2023), Ehrgott (2005): ε-constraints dominate weighted-sum

Quick-Commerce & Indian Context

- Sinchaisri et al. (2023): SLA pressure in q-commerce
- Das et al. (2020): Indian gig-rider compliance

Metaheuristics

- Prins (2004): giant-tour + Bellman–Ford split
- Ropke & Pisinger (2006): Adaptive LNS

Data & Calibration

- Boeing/OSMnx (2017): street networks
- BPR (1964), HCM (2010): congestion modelling
- MoRTH (2022), Mohan et al. (2017): India road safety

Research Gaps

Gap 1
No framework combining safety constraints + soft TW + q-commerce SLA

Gap 2
Crash risk uncalibrated: no reproducible public-source chain for India

Gap 3
Solver comparisons use different evaluation code (apples-to-pears bias)

Gap 4
Residential-street penetration not modelled as an explicit operator knob

Gap 5
No benchmark on real Indian dark-store coordinates: synthetic depots used

$\Downarrow$

This thesis addresses all five gaps in one integrated, reproducible framework.

Contributions

1. **SA-VRPTW Formulation**: crash exposure as hard ε-constraint ($\sum R_{ij} x_{ij}^k \leq \bar{R}$); per-route + fleet-wide residential caps $\bar{H}_k = 100$
2. **BASM Calibration**: reproducible public-source chain (MoRTH 2022 + Mohan et al. 2017 + OSM proxy) using only public data; explicit π_{ij} formula published
3. **Unified Solver Contract**: MILP, GA, ALNS all share identical F_1 evaluation and feasibility checker
4. **Key Empirical Finding**: GA with BF-split outperforms ALNS at $N \geq 100$ in quality AND speed (**325× faster at** $N = 200$)
5. **Five-City Benchmark**: real Blinkit dark-store coordinates (Bengaluru, Delhi, Gurugram, Mumbai, Pune); 428 feasible rows
6. **Fully Reproducible Pipeline**: Hydra config, seeded runs, manifest-checkpointed grid, MIT licence

Problem Formulation: SA-VRPTW

Primary Objective (minimise service-quality cost):

$$F_1 = \sum_k \sum_{i \in C} \Big[w_e \, w_i^k + \big(e^{\beta_{\text{stw}} \tau_i^k} - 1 \big) \Big]$$

- w_i^k: early-arrival idle wait
- τ_i^k: lateness beyond 10-min ETA
- Convex penalty $\Rightarrow$ realistic SLA cost

ε-**Constraints** (hard safety bounds):

$$\sum_{(i,j)} R_{ij} \, x_{ij}^k \le \bar{R} \quad \forall k \quad \text{(crash survival)}$$

$$\sum_{(i,j)} H_{ij} \, x_{ij}^k \le \bar{H}_k \quad \forall k \quad \text{(residential cap)}$$

$$\sum_k \sum_{(i,j)} H_{ij} \, x_{ij}^k \le \bar{H} \quad \text{(fleet-wide)}$$

Super-Graph Arc Quantities

Shortest-time path P_{uv} on street graph:

$$T_{uv} = \sum_{e \in P} t_e$$

$$R_{uv} = \sum_{e \in P} -\ln(1 - r_e)$$

$$H_{uv} = \sum_{e \in P} h_e$$

Key Constants

$Q = 2$ items $\quad T_{\max} = 35$ min

$\bar{H}_k = 100$ edges $\quad \beta_{\text{stw}} = 0.12$

PWL breakpoints: $\{0, 2, 5, 10, 15, 20, 30\}$

Multi-depot coupling, MTZ sub-tour elimination, and arrival-time linking constraints fully stated in Chapter 3.

Data Pipeline and BASM Calibration

OSMnx street graph (drive, bbox)

↓

BPR Congestion: $t_{ij} = t_{ij}^{free}(1 + 0.15(V/C)^4)$

↓

BASM Risk: $r_{ij} = \mathrm{clip}(\lambda \cdot \ell \cdot \sigma \cdot \pi_{ij}, 0, 0.99)$

↓

Blinkit KML → k-means depots

↓

Residential-weighted customer sampling

↓

Dijkstra super-arcs: (T_{uv}, R_{uv}, H_{uv})

BASM Local Proxy:

$$\pi_{ij} = \mathrm{clip}\big(1 + 0.40(2\beta_{ij} - 1) + 0.30 s_{ij} + 0.30 c_{ij},\ 0.5,\ 2.0\big)$$

- β_{ij}: normalised edge betweenness
- s_{ij}: local signal density
- c_{ij}: crossing density

Calibration chain:

- MoRTH 2022: category-level fatality totals
- Mohan et al. 2017: arterial > collector > local
- OSM proxy: **public, no restricted data**

Modelled annual events within ±25% of city aggregate for all 5 cities

Solver Architecture: Unified Evaluation Contract

Instance

Typed dataclass
Depots + Customers
Super-arcs (T, R, H)
Q, $T_{\max}$, $\bar{R}$, $\bar{H}_k$

$\Rightarrow$

MILP

HiGHS/CBC solver
PWL SOS2 penalty
$N \leq 50$ instances

GA

Giant-tour chromosome
Bellman–Ford split
200 generations

ALNS

4 destroy operators
3 repair operators
SA acceptance

$\Rightarrow$

Shared Evaluation Contract

`eval.objective.breakdown()`
Computes identical F_1 for all three solvers — no apples-to-pears bias

`eval.feasibility.validate()`
Strict checker: capacity, $T_{\max}$, $\bar{R}$, $\bar{H}_k$

Result: 428 feasible rows, 0 violations

Every solver sees the same Instance, reports through the same F_1 code, and is judged by the same validator. This makes GA vs. ALNS comparisons apples-to-apples.

Algorithm Design: GA + ALNS Pseudocode

Algorithm 1: GA with Bellman–Ford Split

```
Init P random customer permutations
Decode via BF split; evaluate F_1 + penalty
for g = 1 to G = 200 do
    Carry top-e = 4 elites forward
    p_1, p_2 ← tournament(k = 5)
    Child ← OX1 crossover (p_x = 0.85)
    Apply swap / insert / reverse (p_m = 0.20)
    Decode; evaluate; sort; truncate to P
Validate incumbent via validate()
return s*
```

Algorithm 2: ALNS

```
s_0 ← GreedyConstruct(I)
T_SA ← 0.05 · F_1(s_0)
for t = 1 to 5000 do
    q ← ⌈n · U(0.15, 0.45)⌉
    D ← destroy op ∝ w^d
    R ← repair op ∝ w^r
    s' ← R(D(s, q))
    Accept/reject via SA (η = 0.99975)
    Update σ = (33, 9, 1, 0) weights
Validate; return s*
```

BF split adds 10^9 sentinel for routes violating H_k, $\bar{R}$, or $T_{\max}$: hard constraints never silently broken.

Destroy: random, worst-cost, Shaw relatedness, risk-cluster.

Repair: greedy insert, regret-2, regret-3.

Code Snapshot: Bellman–Ford Split with Hard-Constraint Sentinels

```python
# src/savrptw/solvers/_common.py
HARD_VIOLATION = 1e9

for i in range(n):
    cumulative_load = 0
    for j in range(i + 1, n + 1):
        cumulative_load += customers[j-1].demand
        if cumulative_load > instance.Q:
            break  # capacity prune O(n^2)

        rb = build_route(instance, depot,
                         customers[i:j],
                         penalty_weights=pw)
        # Soft F1 + penalty mass
        route_cost = (rb.f1_contribution
                      + rb.violation_mass)

        # Hard-constraint sentinels
        if rb.total_H > instance.H_cap_route:
            route_cost += HARD_VIOLATION
        if rb.total_R > instance.R_bar:
            route_cost += HARD_VIOLATION
        if rb.duration > instance.T_max:
            route_cost += HARD_VIOLATION

        if cost[i] + route_cost < cost[j]:
            cost[j] = cost[i] + route_cost
            pred[j] = i
```

Why this design?

- Shared by GA, ALNS, and MILP evaluator
- F_1 identical across all three: no apples-to-pears bias
- 10^9 sentinel: BF only selects a violating arc when no feasible split exists
- $O(n^2)$, fast: 26 s at $N = 200$

Project layout:

```
src/savrptw/
  solvers/: milp, ga, alns
  instance/: generator, super_arc
  risk/: basm calibration
  eval/: feasibility, objective
conf/: Hydra YAML configs

scripts/: run_grid.py
```

Experimental Setup

Cities and Dark-Store Coverage

City	Blinkit stores
Bengaluru	113
Delhi	111
Mumbai	96
Gurugram	38
Pune	39 (partial data)
Hyderabad	0 (excluded)

Grid:

- $N \in \{20, 50, 100, 200\}$ customers
- Depots: $\{3, 5, 5, 8\}$ per N
- Up to 10 random seeds per cell; solvers: GA and ALNS
- **428 feasible rows**, 0 violations

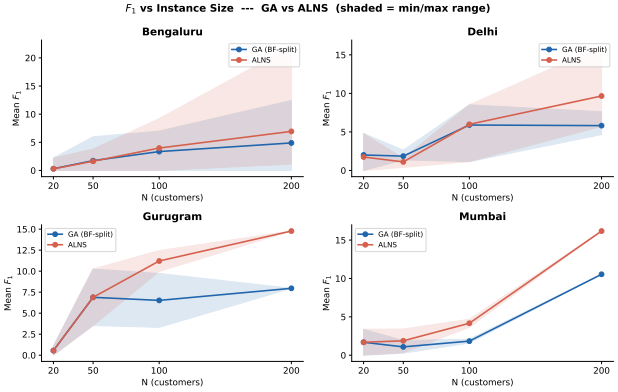
Reproducibility

- Hydra config composition
- Manifest-checkpointed grid runner: crash-safe resumption
- All parameters in `conf/*.yaml`
- BASM guard: refuses uncalibrated source

Evaluation

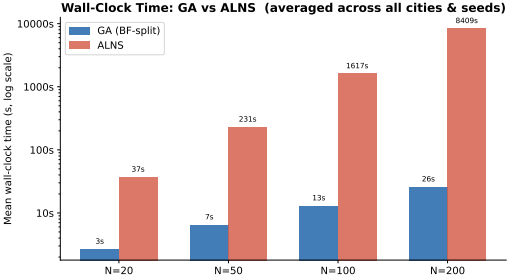
- Strict `validate()` after every solve
- Crash Monte Carlo: 10,000 Bernoulli trials
- Beta-prior compliance sweep: $(\alpha, \beta) \in \{3, 5, 8\} \times \{1, 2, 3\}$

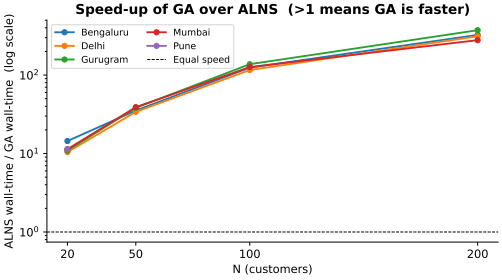
Results: F_1 vs Instance Size (All Cities)



- **GA** and **ALNS** competitive at $N \leq 50$; GA clearly better at $N \geq 100$
- F_1 grows near-linearly with N for both solvers
- Wide bands at Gurugram and Delhi at $N = 200$: seed-sensitive geometry

Results: Wall-Clock Time and Speed-Up Ratio

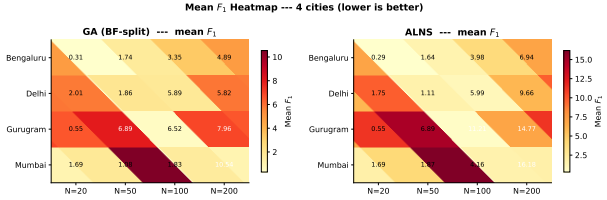


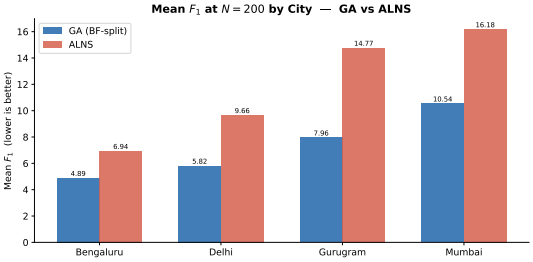


Solver	$N = 20$	$N = 50$	$N = 100$	$N = 200$
GA	2.7 s	6.5 s	13 s	**26 s**
ALNS	31 s	213 s	1669 s	**8407 s**

GA is $\sim$325$\times$ faster at $N = 200$ and produces lower F_1

Results: City-Level Analysis





- **Bengaluru** (113 stores): most stable; lowest F_1; recommended city for sensitivity studies
- **Gurugram & Mumbai**: elevated F_1 at $N \geq 50$ due to fewer depot clusters relative to demand
- **GA** consistently beats **ALNS** across all cities at $N = 200$
- **Delhi** shows widest seed variance at large N: diverse street-network geometry

Limitations & Future Work

Honest Limitations

- BASM is a *calibrated prior*, not a direct measurement: aggregate public data only
- Super-graph fixes paths: optimisation over customer **orderings**, not physical street choices
- MILP scales to $N \leq 50$ only (CBC); commercial solver needed for larger
- Pareto $\bar{R}$, $\bar{H}$ sweep deferred to next run
- Behavioural compliance from single field-data study

Future Directions

1. **Risk-aware path selection**: jointly optimise orderings + physical streets
2. **Pareto sweep**: $\bar{R} \times \bar{H}$ grid traces safety/efficiency frontier
3. **Real-time congestion**: replace static BPR with live feeds
4. **Stochastic demand**: non-stationary order arrivals
5. **Behavioural meta-analysis**: multi-study compliance prior
6. **City-level crash microdata**: upgrade BASM with validated black-spot inventories if available

Conclusion

What We Delivered

- SA-VRPTW formulation with ε-constraints for crash survival and residential-road use
- Reproducible public-source BASM calibration for India: no restricted data needed
- MILP, GA, and ALNS under a unified F_1/feasibility contract
- Empirical benchmark: 428 feasible rows, 5 cities, 4 problem sizes

Key Takeaways

- **ε-constraint framing** gives operators an explicit safety dial: no hidden weight tuning
- **GA with Bellman–Ford split** is recommended: *both* faster and higher-quality than ALNS at $N \geq 100$
- **Calibration honesty** is achievable without restricted microdata: chain is fully reproducible
- Full pipeline released under **MIT licence**: `github.com/Coderopp/MTP-II`

Thank You

Questions & Discussion

Author: Pranav Bhadane (21IM30017)
Supervisor: Prof. Subhajit Sidhanta
Department: Industrial & Systems Engg., IIT Kharagpur
Code: `github.com/Coderopp/MTP-II`

Key references: Solomon (1987) · Prins (2004) · Ropke & Pisinger (2006)
MoRTH (2022) · Mohan et al. (2017) · Boeing/OSMnx (2017)